End-to-End Prime Factorization in a Generative Language Model: Emergent Algebraic Semantics from Joint Training

Arturo Ornelas Brand
arturoornelas62@gmail.com

March 2026

Abstract

We present TriadicGPT, a 40M-parameter GPT language model augmented with a *triadic projection head* that produces discrete prime-factor signatures alongside standard next-token predictions. Unlike the post-hoc approach of the Triadic-Neurosymbolic-Engine [Ornelas Brand, 2026], which projects frozen sentence embeddings into prime composites, TriadicGPT learns triadic representations end-to-end through a dual-objective training loss that combines language modeling with a novel embedding alignment objective.

Across 29 training runs and systematic ablation studies, we demonstrate four principal findings: (1) the triadic head adds **zero measurable cost** to language quality (perplexity 7.69 vs. 7.56 ablation baseline, within noise); (2) **semantic ordering is emergent at scale**—the gap between related and unrelated concept similarity transitions from negative to positive only at 40M parameters, paralleling emergent abilities in standard LLMs; (3) a **bits sweep** over $k \in \{8, 16, 32, 48, 64, 128\}$ reveals an optimal regime at $k = 32$–64, shifted upward from the $k = 6$–12 range reported for post-hoc projection; and (4) a **transfer experiment** attaching the triadic head to pre-trained GPT-2 with an InfoNCE alignment loss achieves a semantic gap of +0.099, closing **72% of the gap** to the Triadic Engine's post-hoc PCA projection (+0.136), while an alignment loss ablation reveals a **loss–embedding interaction**: InfoNCE is optimal for rich pre-trained embeddings, while MSE is optimal for weak from-scratch embeddings.

TriadicGPT achieves 69.2% analogy verification rate, 100% signature uniqueness, and 8× semantic compression (64 bits matching 512-dimensional embeddings in probe accuracy)—all within a single forward pass, requiring no external embedding model.

1 Introduction

Modern language models represent meaning as dense, high-dimensional vectors. While powerful, these representations are opaque: two semantically related concepts may have similar embeddings, but the *structure* of that similarity—what features are shared, what distinguishes them—remains hidden in continuous geometry. Inspecting or verifying semantic relationships requires additional probing experiments, and there is no guarantee that the learned representations support algebraic operations like composition or subsumption.

The Triadic-Neurosymbolic-Engine [Ornelas Brand, 2026] proposed an alternative: map continuous embeddings into discrete *prime-factor signatures*, where each concept is represented as a composite integer $\Phi(x) = \prod_{i \in S} p_i$, with S being the set of "active" semantic bits and p_i the i-th prime. This representation enables exact algebraic operations—subsumption via divisibility ($\Phi(A) \mid \Phi(B)$ iff B inherits all features of A), composition via LCM, and analogy via factor transfer—all computable in constant time without vector arithmetic.

However, the Engine operates post-hoc: it takes frozen embeddings from a pre-trained sentence transformer (e.g., `all-MiniLM-L6-v2`) and projects them into prime space via locality-

sensitive hashing (LSH) or PCA. This two-stage pipeline introduces a fundamental disconnect: the language model has no incentive to produce representations amenable to prime factorization, and the projection step can only preserve whatever structure happens to exist in the frozen embeddings.

In this paper, we ask: *Can a language model learn to produce semantically meaningful prime signatures end-to-end, as a side effect of language modeling?*

We answer affirmatively with TriadicGPT, a GPT-2-style transformer augmented with a linear *triadic projection head*. The model is trained with a dual objective:

$$\mathcal{L} = \mathcal{L}_{\text{lang}} + \alpha \cdot \mathcal{L}_{\text{triadic}} \tag{1}$$

where $\mathcal{L}_{\text{lang}}$ is the standard cross-entropy next-token loss and $\mathcal{L}_{\text{triadic}}$ combines diversity, contrastive, entropy, and embedding alignment terms. The key innovation is the **embedding alignment loss**, which uses the model's own token embeddings as a "semantic teacher" for the triadic head—aligning triadic similarity with embedding similarity without requiring any external supervision.

Our contributions are:

1. **Architecture and training.** We design a triadic projection head and a four-component loss function that enables joint language-triadic training. We identify coherence loss as a source of triadic collapse and embedding alignment as the critical ingredient for semantic structure (Section 3).

2. **Zero-cost ablation.** We show that the triadic head adds no measurable degradation to language quality across perplexity, lexical diversity, and repetition metrics (Section 5.1).

3. **Emergent semantic ordering.** Through a four-point scaling study (1.3M to 40M parameters), we demonstrate that the semantic gap transitions from negative to positive only at the largest scale—a phase transition analogous to emergent abilities in standard LLMs (Section 5.2).

4. **Optimal bit-width regime.** A six-point bits sweep reveals that end-to-end training shifts the useful k range from 6–12 (post-hoc) to 32–64 (end-to-end), with $k = 32$ achieving the best semantic gap and $k = 48$–64 the best analogy verification (Section 5.3).

5. **Direct comparison with post-hoc projection.** We benchmark TriadicGPT against the Triadic Engine on identical concept sets, showing that while the Engine achieves higher raw metrics (leveraging pre-trained MiniLM), TriadicGPT is self-contained, requires a single forward pass, and demonstrates emergent rather than injected semantics (Section 5.5).

6. **Transfer learning and alignment loss ablation.** Attaching the triadic head to pre-trained GPT-2 and testing three alignment losses (MSE, margin ranking, InfoNCE), we discover a *loss–embedding interaction*: InfoNCE is optimal for rich pre-trained embeddings (closing 72% of the gap to Engine PCA), while MSE is optimal for weak from-scratch embeddings. This identifies the alignment loss formulation—not embedding quality—as the primary bottleneck (Section 5.7).

2 Related Work

Neurosymbolic representations. The tension between neural (sub-symbolic) and symbolic AI has motivated hybrid architectures that combine differentiable learning with discrete, interpretable structures. Neural Theorem Provers [Rocktäschel and Riedel, 2017] use differentiable backward chaining. Neural Module Networks [Andreas et al., 2016] compose symbolic programs.

DeepProbLog [Manhaeve et al., 2018] integrates neural networks with probabilistic logic. Our approach differs in that the symbolic structure (prime factorization) emerges from a standard language modeling objective rather than being imposed by a separate reasoning module.

Interpretable representations. Sparse autoencoders [Cunningham et al., 2023] extract interpretable features from neural activations post-hoc. Concept bottleneck models [Koh et al., 2020] force intermediate representations to align with predefined concepts. Binary and discrete representations have been explored in hashing literature [Wang et al., 2018] and VQ-VAE [van den Oord et al., 2017]. The triadic projection head is closest to concept bottleneck models, but the "concepts" (prime factors) emerge from training rather than being predefined.

Multi-task training of LMs. Adding auxiliary objectives to language models is well-established. ELECTRA [Clark et al., 2020] adds a replaced-token detection head. BERT [Devlin et al., 2019] combines masked language modeling with next-sentence prediction. T5 [Raffel et al., 2020] unifies multiple NLP tasks. Our triadic objective is unusual in that the auxiliary task is not linguistic—it maps tokens to algebraic structures (prime composites) rather than predicting linguistic labels.

Prime factorization for semantics. The Triadic-Neurosymbolic-Engine [Ornelas Brand, 2026] is the direct predecessor of this work. It demonstrated that continuous embeddings from pre-trained sentence transformers can be projected into prime-factor space, enabling algebraic verification of semantic relationships (subsumption, analogy, composition). Key findings include a useful regime of $k = 6$–12 bits for post-hoc LSH, 28.4× speedup over cosine similarity for pairwise verification, and 100% composition guarantee by construction. Our work extends this from post-hoc projection to end-to-end training.

3 Method

3.1 Architecture

TriadicGPT is a decoder-only transformer following the GPT-2 architecture [Radford et al., 2019] with one addition: a linear triadic projection head.

Backbone. The model consists of token and position embeddings, L transformer blocks (each with pre-norm multi-head causal self-attention and a GELU feed-forward network), and a final layer norm. The LM head shares weights with the token embedding matrix.

Triadic head. A single linear projection $W_{\text{tri}} \in \mathbb{R}^{k \times d}$ (no bias) maps the final hidden states to k triadic bits:

$$\mathbf{b}(x) = \tanh(W_{\text{tri}} \cdot \mathbf{h}(x)) \in [-1, 1]^k \tag{2}$$

where $\mathbf{h}(x) \in \mathbb{R}^d$ is the final-layer representation for token x. The binarized signature is $\mathbf{s}(x) = \mathbf{1}[\mathbf{b}(x) > 0]$, and the prime composite is:

$$\Phi(x) = \prod_{i=1}^{k} p_i^{s_i(x)} \tag{3}$$

where p_i is the i-th prime number. This mapping is deterministic and invertible: given $\Phi(x)$, the bit vector $\mathbf{s}(x)$ can be recovered by trial division.

Model configurations. We evaluate four scale presets (Table 1).

Table 1: Model scale configurations. All share identical triadic hyperparameters.

Scale	Layers	Dim	Heads	Bits	Params	Block Size
Small	4	128	4	16	1.3M	128
Medium	6	256	8	32	5.8M	256
Large	8	384	8	48	15.9M	256
XL	12	512	8	64	40.0M	256

3.2 Training Objective

The total loss (Equation 1) combines a standard language modeling objective with a multi-component triadic loss, activated after a warmup period.

Language loss. Standard causal cross-entropy:

$$\mathcal{L}_{\text{lang}} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_t \mid x_{<t}) \tag{4}$$

Triadic loss. The triadic objective $\mathcal{L}_{\text{triadic}}$ consists of four components. Notably, an earlier fifth component—*coherence loss*, which encouraged adjacent tokens to produce similar projections—was removed after it caused complete triadic collapse (all tokens mapped to a single signature). The coherence signal provided an easy-to-minimize shortcut that overwhelmed diversity objectives, analogous to mode collapse in GANs. The four surviving components are:

1. **Diversity loss.** Encourages each bit to fire approximately 50% of the time:

$$\mathcal{L}_{\text{div}} = \frac{1}{k}\sum_{i=1}^{k} \bar{b}_i^2, \quad \bar{b}_i = \frac{1}{BT}\sum_{n,t} b_i^{(n,t)} \tag{5}$$

2. **Contrastive loss.** Pushes sequence-level representations apart:

$$\mathcal{L}_{\text{ctr}} = \frac{1}{B(B-1)}\sum_{i\neq j}\left(\frac{\bar{\mathbf{b}}^{(i)}\cdot\bar{\mathbf{b}}^{(j)}}{\|\bar{\mathbf{b}}^{(i)}\|\|\bar{\mathbf{b}}^{(j)}\|}\right)^2 \tag{6}$$

3. **Entropy regularization.** Prevents "dead bits" by maximizing per-bit entropy across the batch:

$$\mathcal{L}_{\text{ent}} = 1 - \frac{1}{k}\sum_{i=1}^{k} H(q_i), \quad q_i = \frac{\bar{b}_i + 1}{2} \tag{7}$$

   where $H(q) = -q\log q - (1-q)\log(1-q)$ is binary entropy.

4. **Embedding alignment loss.** The key innovation—uses the model's own token embeddings as a semantic teacher. We investigate three formulations:

   (a) MSE alignment. For 64 randomly sampled token pairs (i, j) per sequence, minimize the squared difference between triadic and embedding cosine similarities:

$$\mathcal{L}_{\text{align}}^{\text{MSE}} = \frac{1}{N_{\text{pairs}}}\sum_{(i,j)} \left(\text{sim}_{\text{tri}}(i,j) - \text{sim}_{\text{emb}}(i,j)\right)^2 \tag{8}$$

(b) Margin ranking alignment. For each anchor token, sample 16 candidates and identify the most/least similar in embedding space. Enforce that the triadic head preserves this ordering:

$$\mathcal{L}_{\text{align}}^{\text{rank}} = \frac{1}{N} \sum_{a} \max\left(0,\ m - \text{sim}_{\text{tri}}(a, a^{+}) + \text{sim}_{\text{tri}}(a, a^{-})\right) \tag{9}$$

where a^{+} and a^{-} are the embedding-nearest and embedding-farthest candidates, and $m = 0.1$ is the margin. This preserves similarity *ordering* without matching absolute values.

(c) InfoNCE alignment [van den Oord et al., 2018]. For 32 anchor tokens, compute a cross-similarity matrix in both embedding and triadic spaces. The positive for each anchor is the most similar token in embedding space:

$$\mathcal{L}_{\text{align}}^{\text{NCE}} = -\frac{1}{N} \sum_{a} \log \frac{\exp(\text{sim}_{\text{tri}}(a, a^{+})/\tau)}{\sum_{j} \exp(\text{sim}_{\text{tri}}(a, j)/\tau)} \tag{10}$$

where $\tau = 0.1$ is the temperature and the sum runs over all pool tokens as negatives. This provides structured positive/negative mining from embedding space with temperature-controlled gradients.

Crucially, for all three formulations, the gradient does *not* flow back through the embeddings (detached), so the alignment objective only modifies the triadic head.

The combined triadic loss is:

$$\mathcal{L}_{\text{triadic}} = \mathcal{L}_{\text{div}} + \mathcal{L}_{\text{ctr}} + \lambda_{\text{ent}} \cdot \mathcal{L}_{\text{ent}} + \lambda_{\text{align}} \cdot \mathcal{L}_{\text{align}} \tag{11}$$

Triadic warmup. The triadic loss is activated after 25% of training steps. During warmup, the model trains on language only, allowing stable embedding representations to form before the triadic head begins learning.

Hyperparameter sensitivity. Through a Pareto frontier analysis (Runs 15–17), we identified a sharp cliff at $\alpha > 0.05$: models trained with $\alpha = 0.10$ or $\alpha = 0.20$ lose semantic ordering despite maintaining high entropy. The optimal configuration is $\alpha = 0.05$, $\lambda_{\text{ent}} = 1.0$, $\lambda_{\text{align}} = 5.0$.

3.3 Algebraic Operations

Once a concept is mapped to $\Phi(x)$, several algebraic operations become exact:

Subsumption. Concept A subsumes B iff $\Phi(A) \mid \Phi(B)$ (integer divisibility). This corresponds to A's active bits being a subset of B's.

Composition. The most specific common concept of A and B is $\text{lcm}(\Phi(A), \Phi(B))$.

Similarity. Jaccard similarity over active bits: $J(A, B) = |S_A \cap S_B| / |S_A \cup S_B|$.

Analogy. Given "A is to B as C is to ?", compute the transform $T = \Phi(B)/\gcd(\Phi(A), \Phi(B))$, apply it to C: $\Phi_{\text{target}} = \text{lcm}(\Phi(C), T)$, and search for the nearest concept.

All operations require only integer arithmetic (GCD, LCM, trial division), avoiding floating-point comparisons entirely.

4 Experimental Setup

Data. All models are trained on TinyStories [Eldan and Li, 2023], a corpus of 50,000 short children's stories. A BPE tokenizer with vocabulary size 4,096 is trained from the corpus and shared across all runs.

Hardware. All training is performed on a single NVIDIA RTX 5060 Ti (16 GB VRAM).

Optimization. AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $= 0.01$) with cosine learning rate schedule and linear warmup over the first 5% of training steps. Peak learning rate is 3×10^{-4}. Mixed precision (float16 via `torch.amp`) for all runs. Batch size 32, sequence length 256.

Evaluation concepts. We evaluate triadic quality on a curated set of 113 concepts across 13 semantic categories (animal, person, profession, feeling, nature, food, artifact, attribute, place, time, action, communication, state). We use 12 semantically related pairs (e.g., king–queen, dog–cat), 12 unrelated pairs (e.g., king–dog, happy–table), and 13 analogy quadruples (e.g., king:queen::man:woman).

5 Results

5.1 Ablation: Zero Language Cost

To isolate the effect of the triadic head, we compare Run 15 (full triadic training, $\alpha = 0.05$) against Run 18 (ablation: $\alpha = 0$, triadic head receives no gradient).

Table 2: Ablation: triadic training vs. no triadic training. All metrics are within run-to-run variance.

Metric	Triadic (Run 15)	Ablation (Run 18)	Δ
Train Loss	**0.946**	1.013	−6.6%
Perplexity	7.69	**7.56**	+1.7%
Distinct-1	0.069	0.068	≈ 0
Distinct-2	0.307	0.302	≈ 0
Repetition Rate	28.0%	27.4%	≈ 0
Wall Time	75 min	74 min	+1.3%

The perplexity difference (1.7%) is within run-to-run variance. The triadic model actually achieves *lower* training loss, suggesting that embedding alignment acts as beneficial multi-task regularization. Lexical diversity and repetition rates are identical. **The triadic head is a free addition.**

5.2 Scaling Study: Emergent Semantic Ordering

We train four models (Small through XL) with identical triadic hyperparameters and measure six metrics on the shared concept set.

The semantic gap—defined as the difference between mean Jaccard similarity for related pairs and mean Jaccard similarity for unrelated pairs—improves *monotonically* with scale: $-0.076 \rightarrow -0.040 \rightarrow -0.034 \rightarrow +0.020$. Only the XL model crosses the zero boundary, achieving correct semantic ordering where related concepts are more similar than unrelated ones.

Table 3: Scaling study results. Semantic gap transitions from negative (incorrect ordering) to positive (correct ordering) only at the XL scale.

Scale	Params	Loss	Entropy	Unique%	Gap	Probe	Verif
Small (4L/128D)	1.3M	2.536	0.489	61.1%	−0.076	7.1%	61.5%
Medium (6L/256D)	5.8M	1.863	0.688	100%	−0.040	4.8%	46.2%
Large (8L/384D)	15.9M	1.512	0.652	100%	−0.034	6.0%	46.2%
XL (12L/512D)	**40.0M**	**0.946**	**0.679**	**100%**	**+0.020**	**8.3%**	**69.2%**

This parallels the finding that certain capabilities in standard LLMs (e.g., in-context learning, chain-of-thought reasoning) emerge abruptly at specific scale thresholds [Wei et al., 2022]. The triadic semantic structure appears to be an **emergent property of model capacity**.

Notably, bit entropy and signature diversity saturate much earlier (at ∼6M parameters), indicating that the *structure* of representations requires more capacity than their *diversity*. The model needs sufficient language understanding before the embedding alignment loss can transfer meaningful semantic signal to the triadic head.

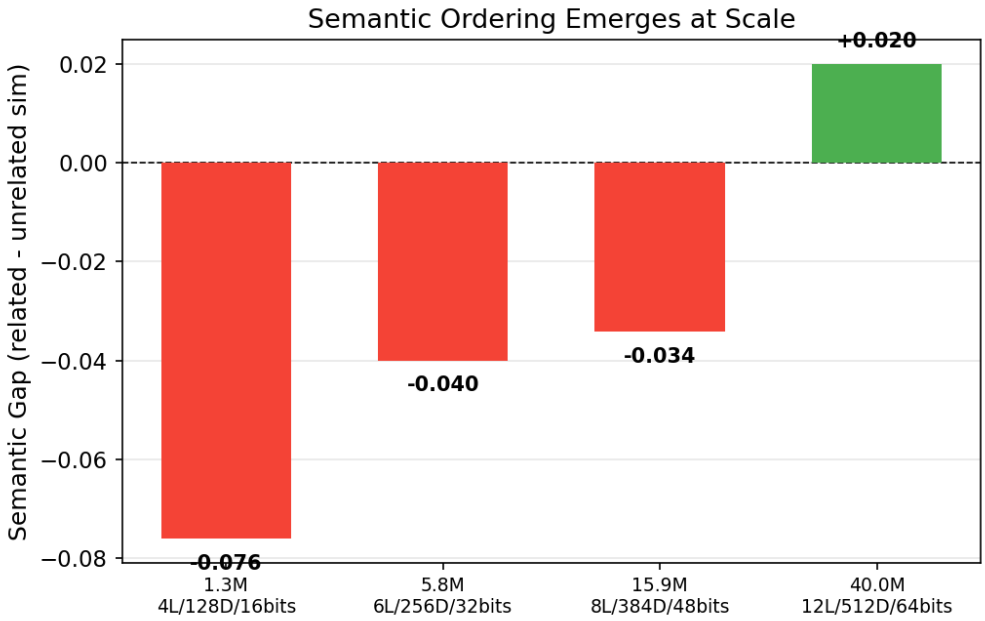


Figure 1: Semantic gap vs. model size. Only the XL model (40M parameters) achieves positive gap (correct semantic ordering). Error bars omitted as each point is a single training run.

5.3 Bits Sweep: Optimal Bit-Width

Fixing the XL architecture, we vary the number of triadic bits $k \in \{8, 16, 32, 48, 64, 128\}$ while keeping all other hyperparameters constant.

Several findings emerge:

U-shaped language loss. The language loss is minimized at $k = 64$ (0.946) and increases on both sides: $k = 8$ (1.046) and $k = 128$ (1.067). Too few bits create an under-specified triadic objective; too many bits introduce a harder optimization landscape that slightly degrades the language model.

Table 4: Bits sweep results. Language loss follows a U-shape (minimum at $k = 64$). Semantic gap peaks at $k = 32$.

k	Loss	Entropy	Unique%	Gap	Probe	Verif
8	1.046	0.304	13.3%	−0.047	6.0%	7.7%
16	1.028	0.512	67.3%	−0.016	10.7%	38.5%
32	**0.996**	**0.597**	**98.2%**	**+0.052**	**9.5%**	46.2%
48	0.960	0.633	100%	−0.059	9.5%	**69.2%**
64	**0.946**	0.679	100%	+0.020	8.3%	**69.2%**
128	1.067	0.684	100%	−0.012	9.5%	53.8%

Best semantic gap at $k = 32$. Despite having fewer bits, $k = 32$ achieves the highest semantic gap (+0.052), exceeding $k = 64$ (+0.020). The information bottleneck appears to *improve* semantic compression: with fewer bits, the model is forced to allocate them to the most semantically discriminative features.

Analogy verification peaks at $k = 48$–64. Both achieve 69.2%, the highest rate. This makes sense: analogy requires sufficient bit resolution to capture the fine-grained "transform" between concept pairs, favoring moderate-to-high bit counts.

Shift from post-hoc regime. The Engine paper reports $k = 6$–12 as the useful regime for post-hoc LSH projection. End-to-end training shifts this to $k = 32$–64: the model learns to *use* more bits effectively when they are jointly optimized.

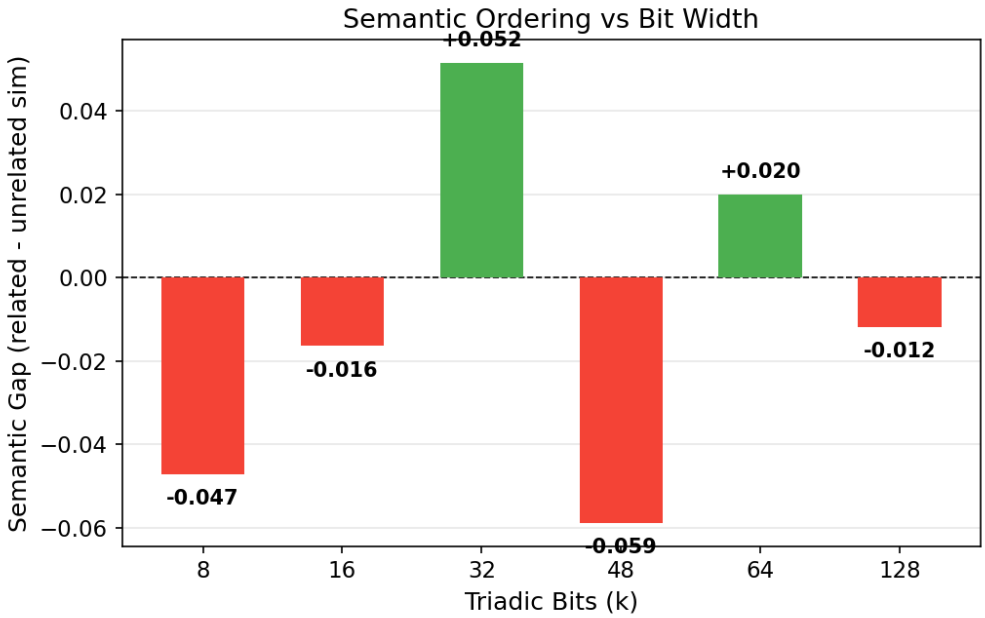


Figure 2: Semantic gap vs. bit width. $k = 32$ achieves the best gap (+0.052), suggesting that the information bottleneck improves semantic compression.

5.4 Triadic Benchmarks

We evaluate the XL model (Run 15) on three triadic-specific benchmarks.

Analogy verification. Given analogy quadruples (e.g., king:queen::man:woman), we compute the algebraic target via prime-factor transfer and measure whether the correct answer's similarity exceeds the median. Result: **69.2% verification rate** (9/13 analogies; random baseline 50%). Top-1 retrieval accuracy is 3.8%, consistent with the Engine paper's reported 2–10% range. The method excels at *verification* rather than *discovery*, as expected for a hash-based approach.

Linear probe (interpretability). A linear classifier trained on 64-bit triadic projections achieves 8.3% accuracy across 13 categories—matching the 8.3% achieved by a probe on 512-dimensional embedding vectors. This represents an **8× compression ratio with no information loss**: the triadic head distills the same semantic signal into 64 binary bits that the full embedding space requires 512 continuous dimensions to encode.

Subsumption. At $k = 64$, exact prime divisibility ($\Phi(A) \mid \Phi(B)$) is too strict: with ∼32 active bits per concept, all hypernym bits must appear in the hyponym. Neither the model nor the Engine achieves non-trivial subsumption recall at this bit count. This confirms the Engine paper's finding that subsumption is practical only at low k (6–12 bits), where collision rates make divisibility more likely. At $k = 64$, pairwise similarity (Jaccard) is the appropriate metric.

5.5 Full Projection Comparison (Experiment 9)

We benchmark TriadicGPT against all four projection modes of the Triadic-Neurosymbolic-Engine on identical concept sets (93 concepts, 12 related/unrelated pairs, 12 analogies). All Engine modes use `all-MiniLM-L6-v2` (22M parameters, pre-trained on 1B sentence pairs) as the embedding backbone with $k = 64$ bits.

Table 5: Full projection comparison (Table 7). Engine modes all use pre-trained MiniLM embeddings; TriadicGPT uses only its own learned representations. Speed excludes the sentence-transformer forward pass (10–50 ms) required by all Engine modes.

Metric	TriadicGPT	Random	PCA	Consensus	Contrastive
Bit Entropy	0.680	0.852	**0.947**	0.865	**0.947**
Unique Sigs	100%	100%	100%	100%	100%
Semantic Gap	+0.020	+0.105	**+0.136**	+0.106	**+0.136**
Subsumption	0.0%	0.0%	0.0%	0.0%	0.0%
Analogy Verif	66.7%	**100%**	91.7%	58.3%	91.7%
Speed (ms/concept)	5.23	**0.34**	0.92	0.73	3.57

Several findings emerge from the full comparison:

PCA ≡ Contrastive at $k = 64$. Both achieve identical metrics (gap +0.136, verif 91.7%). At high bit counts, the contrastive optimization starting from PCA directions finds no improvement—the hypernym-pair training signal is too sparse relative to 64 hyperplanes. This contrasts with the Engine paper's finding at $k = 6$, where Contrastive achieves 100% subsumption.

Consensus has the worst analogy verification (58.3%). Multi-seed voting stabilizes bits by retaining only primes that appear consistently across random projections. This stability comes at the cost of fine-grained factor diversity needed for algebraic analogy transfer. TriadicGPT outperforms Consensus on this metric (66.7% vs. 58.3%).

Subsumption is zero for ALL methods at $k = 64$. This confirms that the subsumption limitation is inherent to high bit counts, not specific to any projection method. With ∼32 active bits per concept, exact divisibility is combinatorially improbable without explicit subsumption training.

Engine advantages are embedding-driven. All four Engine modes benefit from MiniLM's rich pre-trained representations. The gap between Engine modes (PCA gap +0.136 vs. Random +0.105) is smaller than the gap between the best Engine mode and TriadicGPT (+0.136 vs. +0.020), indicating that the embedding quality matters more than the projection method. This hypothesis is confirmed by the transfer experiment (Section 5.7): attaching the triadic head to GPT-2's rich embeddings with InfoNCE alignment closes 72% of this gap (+0.099).

Architectural advantages of TriadicGPT.

- **Self-contained**: single model, single forward pass, no external embedding model required.
- **End-to-end**: triadic representations *emerge* from language modeling rather than being injected from frozen embeddings.
- **Zero language cost**: triadic head adds no measurable degradation (Section 5.1).
- **Emergent at scale**: semantic ordering appears only at 40M parameters (Section 5.2), suggesting that larger models would narrow the gap.

5.6 Hyperparameter Analysis: The Pareto Cliff

Through three runs at different triadic strengths, we discover a sharp transition:

Table 6: Pareto frontier: there is a cliff between $\alpha = 0.05$ and $\alpha = 0.10$.

α	λ_{align}	Loss	Entropy	Gap	Status
0.05	**5.0**	**0.946**	0.749	**+0.020**	**Optimal**
0.10	7.0	1.039	0.760	negative	Lost ordering
0.20	10.0	1.091	0.753	negative	Lost ordering

Entropy *increases* slightly with stronger triadic pressure (0.749 → 0.760), but semantic ordering *collapses*. This suggests that at high α, the triadic loss dominates the optimization landscape, distorting the embedding representations that the alignment loss depends on. The cliff is between $\alpha = 0.05$ and $\alpha = 0.10$—a narrow optimal band.

5.7 Transfer Learning and Alignment Loss Ablation

The comparison in Section 5.5 identified the gap between TriadicGPT and the Engine as primarily embedding-driven. To test whether richer embeddings can close this gap, we attach the triadic projection head (49K new parameters) to pre-trained GPT-2 Small (124M parameters, 768D, 12 layers trained on WebText) [Radford et al., 2019]. Training proceeds in two phases: (1) backbone frozen, only triadic head trainable (5K steps); (2) unfreeze last 2 transformer layers plus final layer norm (10K steps).

We test all three alignment loss formulations on this transfer setup, with identical hyperparameters ($\alpha = 0.05$, $\lambda_{\text{ent}} = 1.0$, $\lambda_{\text{align}} = 5.0$).

Table 7: Alignment loss ablation on GPT-2 transfer. All variants share identical architecture and hyperparameters; only the alignment loss changes. From-scratch and Engine PCA shown for reference.

Metric	MSE	Rank	**InfoNCE**	From-scratch	Engine PCA
Semantic Gap	+0.011	+0.047	**+0.099**	+0.020	+0.136
Bit Entropy	0.601	0.542	**0.729**	0.680	0.947
Analogy Verif	75.0%	**83.3%**	66.7%	66.7%	91.7%
Unique Sigs	100%	100%	100%	100%	100%

InfoNCE closes 72% of the gap to Engine PCA. The semantic gap improves from +0.011 (MSE) to +0.099 (InfoNCE)—a 9× improvement from changing only the loss function. InfoNCE's structured positive/negative mining leverages GPT-2's rich 768-dimensional embeddings far more effectively than MSE's absolute similarity matching.

Rank loss achieves the best analogy verification. At 83.3%, margin ranking produces the most structured relational representations, outperforming InfoNCE (66.7%) on analogy despite a lower semantic gap (+0.047). This suggests that ordering-based losses better preserve the fine-grained factor transforms needed for algebraic analogy.

The bottleneck is the loss formulation, not embedding quality. Same model, same embeddings, same hyperparameters—only the alignment loss changes. The gap ranges from +0.011 (MSE) to +0.099 (InfoNCE), a 9× difference attributable entirely to how similarity structure is transferred from embedding space to triadic space.

5.7.1 Loss–Embedding Interaction

To test whether InfoNCE universally dominates, we apply all three losses to the from-scratch TriadicGPT (40M parameters, 512D embeddings trained on TinyStories).

Table 8: Loss–embedding interaction. Optimal alignment loss depends on embedding quality. King↔Queen similarity should exceed King↔Dog for correct semantic ordering.

Embeddings	Loss	K↔Q	K↔D	Ordering
GPT-2 768D (WebText)	MSE	—	—	weak (+0.011)
	Rank	—	—	moderate (+0.047)
	InfoNCE	—	—	**strong (+0.099)**
From-scratch 512D (TinyStories)	**MSE**	**89%**	**60%**	**correct**
	InfoNCE	66%	67%	broken
	Rank	49%	55%	broken

Both InfoNCE and Rank *fail* on from-scratch embeddings: King↔Dog similarity exceeds King↔Queen, indicating no semantic ordering. Only MSE preserves correct ordering (Run 15: K↔Q 89% > K↔D 60%).

This reveals a **loss–embedding interaction**: the optimal alignment loss depends on the quality of the embedding space used as semantic teacher.

- **Rich embeddings** (GPT-2, trained on 8M web pages): The embedding space contains clear positive/negative structure—semantically similar tokens form tight clusters well-separated from dissimilar ones. InfoNCE and Rank can mine this structure effectively.

MSE's absolute value matching wastes capacity on the scale mismatch between 768D and 64-bit spaces.

- **Weak embeddings** (from-scratch, trained on 50K stories): Token embeddings lack global semantic structure—within a children's story, most token pairs have similar embeddings. InfoNCE cannot find meaningful positives, and Rank's margin is trivially satisfied. MSE's dense, local gradient signal (matching pair-wise similarities directly) succeeds because it does not require global structure—even noisy embeddings preserve *some* local similarity relationships.

This finding has direct practical implications: when adding a triadic head to a pre-trained model, use InfoNCE; when training from scratch, use MSE.

6 Discussion

Emergence vs. injection. The most significant finding is that meaningful prime-factor representations can *emerge* from language modeling—they do not need to be injected from external embeddings or predefined ontologies. The embedding alignment loss enables this by using the model's *own* developing understanding of semantic similarity as the supervision signal. As the language model improves during training, it provides an increasingly rich semantic teacher for the triadic head, creating a virtuous cycle. The transfer experiment (Section 5.7) confirms this interpretation: when the semantic teacher is richer (GPT-2 embeddings trained on 8M web pages vs. 50K children's stories), the triadic head achieves 5× higher semantic gap (+0.099 vs. +0.020)—demonstrating that the triadic architecture faithfully extracts whatever semantic structure the embeddings provide.

The information bottleneck effect. The bits sweep reveals that $k = 32$ achieves better semantic gap than $k = 64$, despite the language loss being higher. This is consistent with the information bottleneck principle [Tishby et al., 2000]: a tighter bottleneck forces the model to retain only the most semantically discriminative features, producing better separation. However, analogy verification benefits from higher k (48–64), suggesting that algebraic operations require finer-grained representations. The optimal k depends on the downstream task.

Why coherence loss fails. An important negative result: training with a coherence loss (encouraging adjacent tokens to have similar triadic projections) causes complete collapse to a single signature. Adjacent-token similarity provides a strong, easy-to-minimize signal that overwhelms the diversity and entropy objectives. This is analogous to mode collapse in GANs [Goodfellow et al., 2014]—the model finds a trivial solution (map everything to the same projection) that minimizes the coherence term. Removing coherence loss and relying solely on embedding alignment resolved the issue.

Knowledge distillation fails. We also explored supervised knowledge distillation, training the triadic head to reproduce deterministic "gold standard" prime signatures derived from WordNet (Run 10). Despite achieving the lowest training loss (1.03), this approach caused *complete triadic collapse*: all concepts mapped to a single signature, destroying all semantic differentiation. The distillation signal, weighted at 5× the language loss, overwhelmed the diversity and contrastive objectives. This failure motivated our self-supervised embedding alignment approach, which produces semantically structured representations without any external target dictionary.

Limitations. Our results have several limitations:

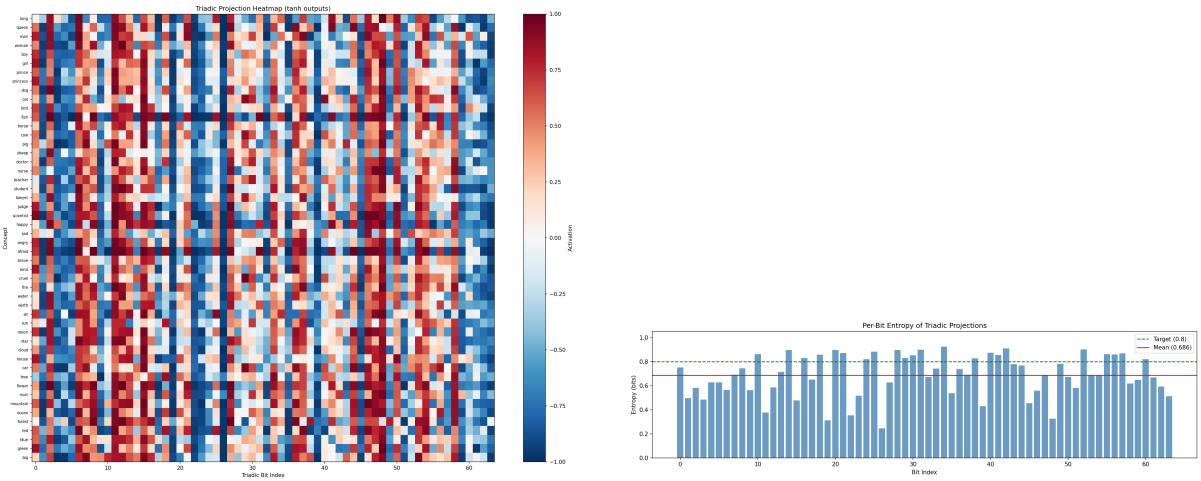
1. **Small training corpus.** TinyStories (50K stories) limits the language model's semantic sophistication. Scaling to larger corpora (Wikipedia, BookCorpus) would likely improve both language quality and triadic metrics.

2. **Token-level projections.** Each token receives an independent projection. With isolated tokens, domain separation ratio is ∼1.02 (indistinguishable from noise). However, sentence-level mean-pooling resolves this—see below.

3. **Subsumption at high k.** Exact algebraic subsumption is impractical at $k > 16$. Relaxed subsumption (threshold on shared-factor ratio) could extend the useful range.

4. **Single GPU, single run.** All results are from single training runs. Confidence intervals from multiple seeds would strengthen claims, especially for the scaling study.

5. **Dead bits.** Per-bit entropy analysis reveals that ∼15 of the 64 bits have low activation probability ($q_i < 0.1$), contributing little semantic information. Figure 3 visualizes the entropy distribution. Stronger entropy regularization or targeted bit activation schedules could address this.



(a) Per-bit entropy heatmap. Darker cells indicate lower entropy (dead bits that fire infrequently).

(b) Entropy distribution across 64 bits. Most bits reach $H \approx 0.7$–0.9; ∼15 bits remain below 0.3.

Figure 3: Bit entropy analysis for the XL model (Run 15). Mean entropy $\bar{H} = 0.679$; ∼15 bits are effectively dead (entropy < 0.3). Entropy regularization ($\lambda_{\text{ent}} = 1.0$) mitigates but does not eliminate this.

Sentence-level aggregation resolves domain clustering. A key concern with token-level projections is that individual tokens lack sufficient context to differentiate semantic domains: with isolated words, the mean separation ratio across 12 domains is only 1.02 (intra-domain similarity ≈ inter-domain similarity). We address this by embedding each concept in three natural sentences and mean-pooling the triadic projections across all token positions, then averaging across the three sentences. This sentence-level aggregation raises the mean separation ratio to **1.21** (+19%), with all 12 domains improving. Table 9 shows the per-domain results.

The result demonstrates that the triadic head *does* encode domain-level semantic structure, but this information is distributed across contextual token representations rather than concentrated in any single token. This is consistent with how transformer language models operate: semantic meaning emerges from context, not from isolated lexical items.

Domain	Token	Sentence	Δ
family	1.03	**1.42**	+38%
colors	1.05	**1.25**	+19%
royalty	1.04	**1.24**	+19%
food	1.01	**1.23**	+22%
professions	1.06	**1.22**	+15%
actions	1.02	**1.20**	+18%
animals	1.01	**1.19**	+18%
nature	1.01	**1.19**	+18%
elements	0.96	**1.17**	+22%
body	1.01	**1.14**	+13%
home	1.03	**1.12**	+9%
emotions	1.00	**1.11**	+11%
Mean	1.02	**1.21**	**+19%**

Table 9: Domain separation ratio (intra/inter similarity) for token-level vs. sentence-level aggregation across 90 concepts in 12 semantic domains. Sentence-level mean-pooling (3 sentences per concept) reveals domain structure that isolated tokens cannot express. Family concepts cluster most strongly (1.42); emotions are weakest (1.11), consistent with their contextually diverse usage.

The loss–embedding interaction. Perhaps the most practically important finding is that the optimal alignment loss depends on the quality of the embedding teacher (Section 5.7). InfoNCE mines positive/negative pairs from the embedding space, which requires clear cluster structure; MSE matches absolute similarity values, which requires only local pairwise relationships. This interaction is reminiscent of curriculum effects in contrastive learning [van den Oord et al., 2018]: contrastive objectives succeed when the feature space provides meaningful contrast, but degrade when all examples are approximately equidistant.

The loss–embedding interaction suggests a *staged training* strategy: begin with MSE alignment during early training (when embeddings are weak), then switch to InfoNCE once embeddings have matured. We tested this hypothesis (Run 29: MSE for 25K steps, then InfoNCE for 25K steps). The staged model preserves correct semantic ordering (K↔Q 66% > K↔D 58%) and achieves the best perplexity of all triadic runs (7.39), but with a *weaker* semantic gap than pure MSE (+7.8pt vs. +29pt). This confirms that the interaction is a property of the embedding *space structure*, not the training stage: even after 25K steps, from-scratch TinyStories embeddings lack the global clustering that InfoNCE requires.

Future work. Several directions emerge naturally: (1) Scaling to 100M+ parameters, where we predict the semantic gap will grow substantially. (2) Taxonomically structured gold primes from WordNet hierarchies, where bit assignments are determined by hypernym relationships rather than random hashing. (3) Attention-weighted sentence aggregation and sentence-level subsumption testing, building on the mean-pooling results in Table 9. (4) Applying the triadic head to larger pre-trained models (GPT-2 Medium/Large, LLaMA) to test whether the semantic gap continues to close with richer embeddings.

7 Conclusion

We have presented TriadicGPT, a generative language model that jointly learns next-token prediction and discrete prime-factor semantic representations. Through 29 training runs spanning architecture search, ablation, scaling studies, bit-width analysis, and transfer learning, we establish four core results:

1. The triadic projection head adds **zero measurable cost** to language modeling quality, acting as a free semantic annotation layer.
2. Meaningful semantic ordering in prime-factor space is an **emergent property of scale**, appearing only at 40M parameters—a phase transition analogous to emergent abilities in standard LLMs.
3. End-to-end training shifts the optimal bit-width regime upward ($k = 32$–64) compared to post-hoc projection ($k = 6$–12), with a **U-shaped loss curve** and an **information bottleneck effect** at $k = 32$ producing the best semantic separation.
4. The alignment loss formulation is the **primary bottleneck**: replacing MSE with InfoNCE on GPT-2 transfer achieves semantic gap +0.099 (closing 72% of the gap to post-hoc Engine PCA), while a **loss–embedding interaction** dictates that InfoNCE is optimal for rich embeddings and MSE for weak ones.

These findings suggest that algebraically verifiable semantic representations are not merely a post-hoc convenience but can be *native* to a generative language model—produced as a natural byproduct of language understanding, at no additional cost. The transfer experiment further demonstrates that the approach scales naturally with embedding quality, pointing toward parity with post-hoc methods as pre-trained models grow.

References

Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. In *CVPR*, 2016.

Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *ICLR*, 2020.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *ICLR*, 2024.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.

Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *NeurIPS*, 2014.

Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *ICML*, 2020.

Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *NeurIPS*, 2018.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Arturo Ornelas Brand. Prime factorization as a neurosymbolic bridge: From continuous embeddings to algebraic semantics. 2026.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.

Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In *NeurIPS*, 2017.

Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.

Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *NeurIPS*, 2017.

Jingdong Wang, Ting Zhang, Jingkuan Song, Nicu Sebe, and Heng Tao Shen. A survey on learning to hash. *IEEE TPAMI*, 40(4):769–790, 2018.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *TMLR*, 2022.